Lab 6

For loops in C++

Learning Objectives

After completing this lab, after completing this lab we learnt:

- Understand the concept of repetition (iteration) in programming using loops.
- Learn the structure and syntax of the for loop in C++.
- Write C++ programs using for loops to perform repetitive tasks efficiently.
- Execute counting operations such as printing numbers in sequence using a for loop.
- Apply for loops to solve basic problems like summation, multiplication tables, and patterns.
- Understand how initialization, condition, and increment/decrement work inside a for loop.
- Compare for loops with while and do-while loops in terms of usage and efficiency.

For Loop

A for loop is a control structure in C++ used to execute a block of code repeatedly for a fixed number of times. It is mainly used when the number of iterations is known in advance. The loop consists of three parts: initialization, condition, and increment/decrement.

For Loop Structure (Syntax):

```
for(initialization; condition; increment/decrement) {
    // code to be executed
}
```

Explanation:

- The initial step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the condition is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement just after the for loop.
- After the body of the for loop executes, the flow of control jumps back up to the increment statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
- The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the for loop terminates.

Example:

Prints numbers from 1 to 5 using a while loop.

```
#include <iostream>                  // Input/output library
using namespace std;                 // Standard namespace
int main() {
   for(int i = 1; i <= 10; i++) {    // Loop from 1 to 10
      cout << i << endl;             // Print current value of i
   }     return 0; }                 // End program
```

Output:

1

2

3

4

5

6

7

8

9

10

Example: C++ program using a for loop to check whether a number is prime or

```
#include <iostream>            // Input/output library
using namespace std;           // Standard namespace
int main() {
   int num;                    // Variable to store user input
   bool isPrime = true;        // Assume number is prime initially
   cout << "Enter a number: ";
   cin >> num;                 // Take input from user
   // 0 and 1 are not prime numbers
   if (num <= 1) {
      isPrime = false;
   }
   // Check divisibility using for loop
   for (int i = 2; i < num; i++) {
      if (num % i == 0) {      // If divisible by any number
         isPrime = false;      // Not a prime number
         break;                // Exit loop early
      }
   }
   // Display result
   if (isPrime) {
      cout << num << " is a Prime number." << endl;
   } else {
      cout << num << " is NOT a Prime number." << endl;
   }
   return 0;}                  // End program
```

Nested for Loop:

A nested loop in C++ is a loop placed inside another loop**.** The inner loop runs completely every time the outer loop runs once.

Syntax:

```
for(initialization; condition; increment) {  //Outer loop


  for(initialization; condition; increment){ //inner loop
     // code
  }
```

In nested loops:

- the outer loop controls rows
- the inner loop controls columns

Nested loops are commonly used for:

- patterns
- matrices
- tables

Examples:

```
#include <iostream>
using namespace std;
int main() {
   for(int i = 1; i <= 3; i++) {   //Outer for loop
      for(int j = 1; j <= 2; j++) {  //Inner for loop
         cout << "i = " << i << " , j = " << j ;}}  return 0 };
```

Output:

i = 1 , j = 1

i = 1 , j = 2

i = 2 , j = 1

i = 2 , j = 2

i = 3 , j = 1

i = 3 , j = 2

Lab5: Lab Tasks

Task 1

• Write a C++ program that uses a while loop to display numbers from 1 to 10

• Display each number on a separate line.

```
#include <iostream>            // Input/output library
using namespace std;           // Standard namespace
int main() {
   for (int i = 1; i <= 10; i++) {   // For loop from 1 to 10
      cout << i << endl;          // Print number on new line
   }
   return 0;                      // End program
}
```

Task 2: Nested For loop

Multiplication Table 1-10

```
#include <iostream>      // Input/output library
using namespace std;     // Standard namespace
int main() {
  for (int i = 1; i <= 10; i++) {      // Outer loop for rows (1 to 10)
    for (int j = 1; j <= 10; j++) { // Inner loop for coluns (1 to 10)
        cout << i * j << "\t";       // Print multiplication value
      }
      cout << endl;   // Move to next line after each row
   }
   return 0;           // End program
}
```

Task 3: Sum of numbers Using for loop

```
#include <iostream>      // Input/output library
using namespace std;     // Standard namespace
int main() {
   int num;              // User input
   int sum = 0;          // Store sum
   cout << "Enter a positive integer: ";   // Ask user
   cin >> num;           // Read input
   for (int i = 1; i <= num; i++) {        // Loop from 1 to num
      sum = sum + i;   // Add each number to sum
   }
   cout << "Sum = " << sum << endl;        // Display result
   return 0;             // End program
}
```

Task 4: Nested for loop Triangle pattern

```
#include <iostream>                    // Input/output library
using namespace std;                   // Standard namespace
int main() {
    int rows;                          // Number of rows
    cout << "Enter number of rows: ";  // Ask user
    cin >> rows;                       // Read input
    for (int i = 1; i <= rows; i++) {  // Outer loop for rows
        for (int j = 1; j <= i; j++) {  // Inner loop for stars
            cout << "* "; }             // Print star
        cout << endl;   }              // Move to next line
    return 0; }                        // End program
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```